

Language Models



Probabilistic models

Statistical Language Model

A statistical language model is a *probability distribution* P over sequences of terms.

Given a document d that is composed of a sequence of words $w_1w_2w_3$, we can define:

$$P(d) = P(w_1w_2w_3) = P(w_1)P(w_2 | w_1)P(w_3 | w_1w_2)$$

Depending on the assumptions we make on the probability distribution, we can create statistical model of different complexity.

The formula above makes no assumptions and can exactly model any language, yet it is impractical because it requires to learn the probability of any sequence in the language.

Unigram model

A unigram model assumes a *statistical independence* between words, i.e., the probability of d is the product of the probabilities of its words:

$$\begin{aligned} P(d) &= P(w_1 w_2 w_3) = P(w_1) P(w_2 | w_1) P(w_3 | w_1 w_2) \\ &= P(w_1) P(w_2) P(w_3) = \prod_i P(w_i) \end{aligned}$$

The bayesian classifier that uses this model is called *naïve* for this reason. Usually the models use the logs of the probabilities to work in a linear space:

$$\log(\prod_i P(w_i)) = \sum_i \log(P(w_i))$$

Smoothing, e.g., add one to all frequencies, is used to avoid zero probabilities.

Bigram/n-gram model

A bigram model assumes a *statistical dependence* of a word from the preceding one:

$$\begin{aligned}P(d) &= P(w_1 w_2 w_3) = P(w_1)P(w_2 | w_1)P(w_3 | w_1 w_2) \\ &= P(w_1)P(w_2 | w_1)P(w_3 | w_2) = \prod_i P(w_i | w_{i-1})\end{aligned}$$

This simple addition is already able to capture a good amount of language regularities.

In general, the longer the **n-gram** we adopt for the model:

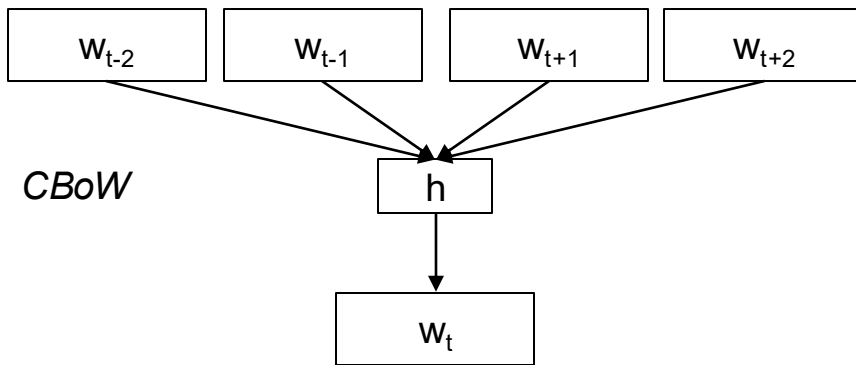
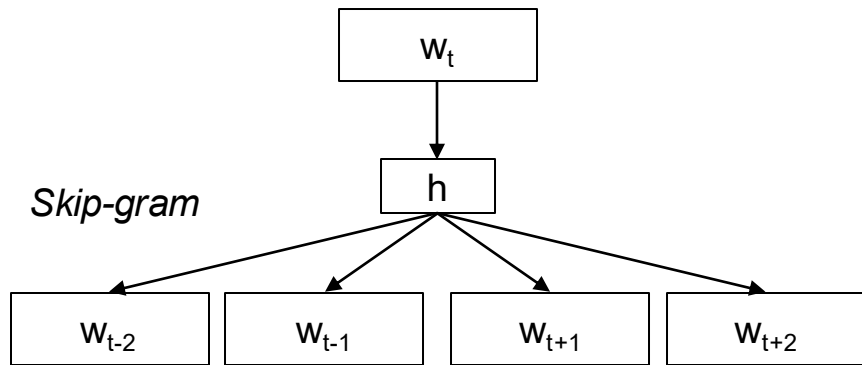
- the more semantic is captured;
- the less statistical significant is the model (memorization/generalization).

Neural models

Word2Vec

Skip-gram and CBoW models of Word2Vec define tasks of **predicting** a *context* from a word (Skip-gram) or a *word* from its *context* (CBoW).

They are both implemented as a two-layers linear **neural network** in which input and output words one-hot representations which are encoded/decoded into/from a dense representation of smaller dimensionality.



Word2Vec

Embeddings are a byproduct of the word prediction task.

Even though it is a prediction task, the network can be trained on any text, no need for human-labeled data!

The context window size ranges between two and five words before and after the central word.

Longer windows capture more semantic, less syntax (typically 6-10 words).

A typical size for the resulting embedding vectors is 200~300.

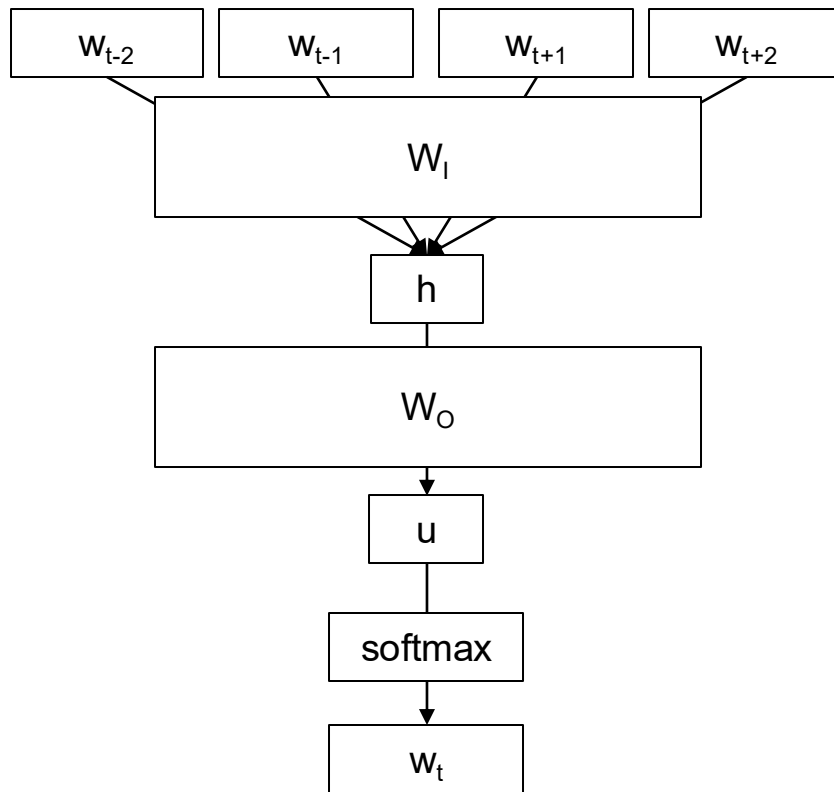


Word2Vec

The Word2Vec network has two layers.

W_I takes in input the sparse representation of the words (one-hot vector of size $|F|$) and produce an abstract representation h of the context.

W_O converts h into a probability vector u that is the prediction of the target word.

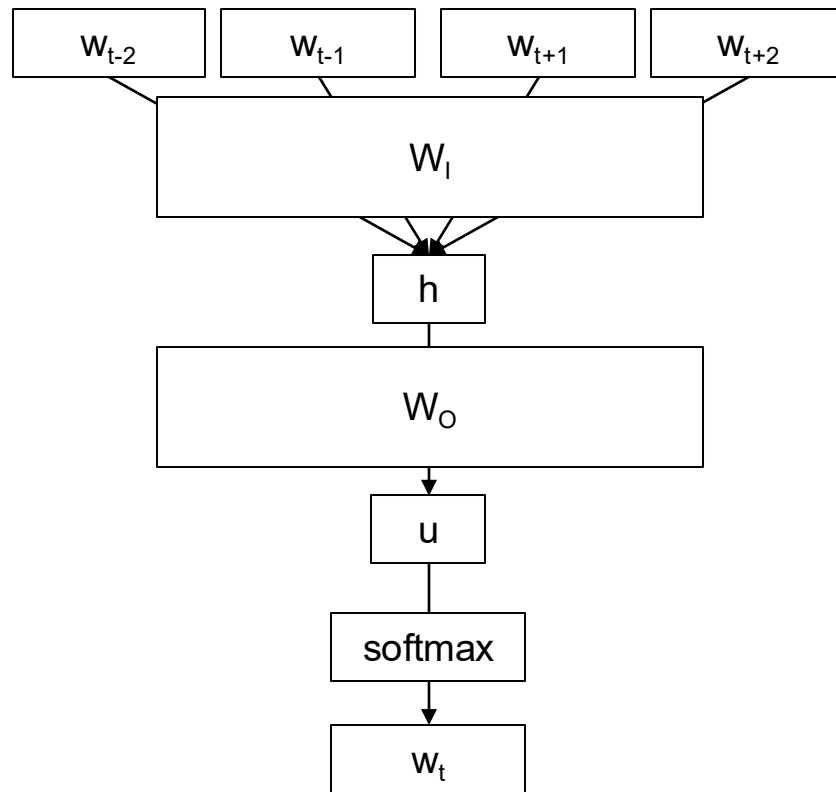


Word2Vec

Every row of W_I is the actual embedding, that is fit during the training process of the prediction task.

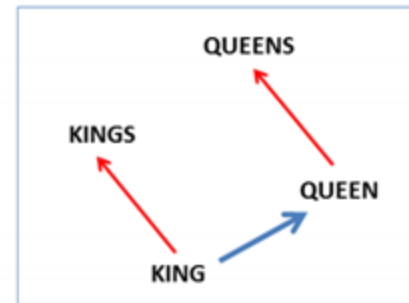
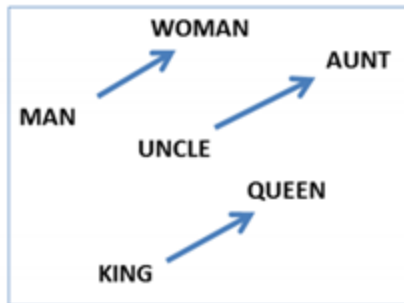
Words that have a similar use in language end up with similar embedding vectors.

Different aspects of language are implicitly modeled into different dimensions of the embeddings.



Testing embeddings

Testing if embeddings capture syntactic/semantic properties.



Analogy test:

Paris	stands to	France	as	Rome	stands to	?
Writer	stands to	book	as	painter	stands to	?
Cat	stands to	cats	as	mouse	stands to	?

$$e('France') - e('Paris') + e('Rome') \sim e('Italy')$$

a : b = c : d

$$d = \arg \max_x \frac{(e(b) - e(a) + e(c))^T e(x)}{\|e(b) - e(a) + e(c)\|}$$

The impact of training data

The source on which a model is trained determines what semantic is captured.

WIKI	BOOKS	WIKI	BOOKS
sega		chianti	
dreamcast	motosega	radda	merlot
genesis	seghe	gaiole	lambrusco
megadrive	seghetto	montespertoli	grignolino
snes	trapano	carmignano	sangiovese
nintendo	smerigliatrice	greve	vermentino
sonic	segare	castellina	sauvignon

Computing embeddings

[Gensim](#) provides an efficient and detailed implementation.

```
sentences = [['this', 'is', 'a', 'sentence'],  
              ['this', 'is', 'another', 'sentence']]
```

```
from gensim.models import Word2Vec  
model = Word2Vec(sentences)
```

[This](#) is a from scratch implementation of W2V skip-grams using pytorch.

FastText word representation

FastText extends the W2V embedding model to ngrams of the words.

The word "goodbye" is also represented with a set of ngrams, including start of word and end of word ('<' and '>':

"<go", "goo", "ood", "odb", "dby", "bye", "ye>"

The length of the ngram is a parameter.

Typically all ngrams of length from 3 to 6 are included.

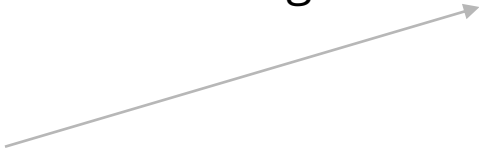
FastText word representation

The embedding of a word is determined as the sum of the embedding of the word and of the embedding of its ngrams.

Subword information allows to give an embedding to OOV words.

Subword information improves the quality of misspelled words.

[Pretrained embeddings for 200+ languages.](#)



```
Query word? gearshift
gearing 0.790762
flywheels 0.779804
flywheel 0.777859
gears 0.776133
driveshafts 0.756345
driveshaft 0.755679
daisywheel 0.749998
wheelsets 0.748578
epicycles 0.744268
gearboxes 0.73986
```



```
Query word? accomodation
sunnhordland 0.775057
accomodations 0.769206
administrational 0.753011
laponian 0.752274
ammenities 0.750805
dachas 0.75026
vuosaari 0.74172
hostelling 0.739995
greenbelts 0.733975
asserbo 0.732465
```

```
Query word? accomodation
accomodations 0.96342
accommodation 0.942124
accommodations 0.915427
accommodative 0.847751
accommodating 0.794353
accomodated 0.740381
amenities 0.729746
catering 0.725975
accomodate 0.703177
hospitality 0.701426
```

Word embeddings to documents

How can we represent *a document* in an embedding space?

A document is composed by a set of words, so we can take their embeddings and combine them in some way to obtain a single document embedding vector (e.g., by taking the average/max of all the word embeddings).

We can extend the Word2Vec model to explicitly model the embeddings of document, e.g., the **Doc2Vec** model.

We can directly use the word embeddings as the first layer in a neural network, letting the neural network learn to implicitly model the content of the document.

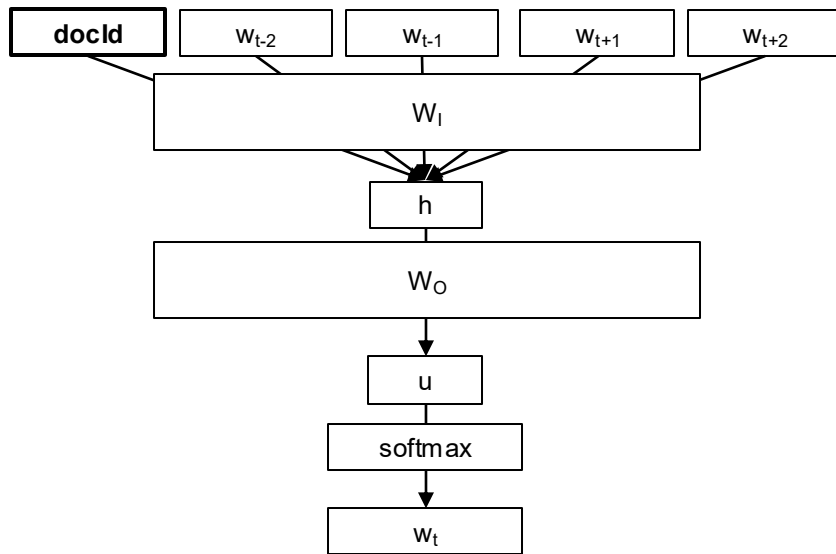
Doc2Vec

Proposed by [Le and Mikolov](#), Doc2Vec extends Word2Vec by adding input dimensions for identifiers of documents.

W_I matrix is $(|D| + |F|) \cdot |h|$.

Documents ids are projected in the same space of words.

The trained model can be used to infer document embeddings for previously unseen documents - by passing the words composing them.



Embeddings in neural networks

An embedding layer in neural networks is typically the first layer of the network.

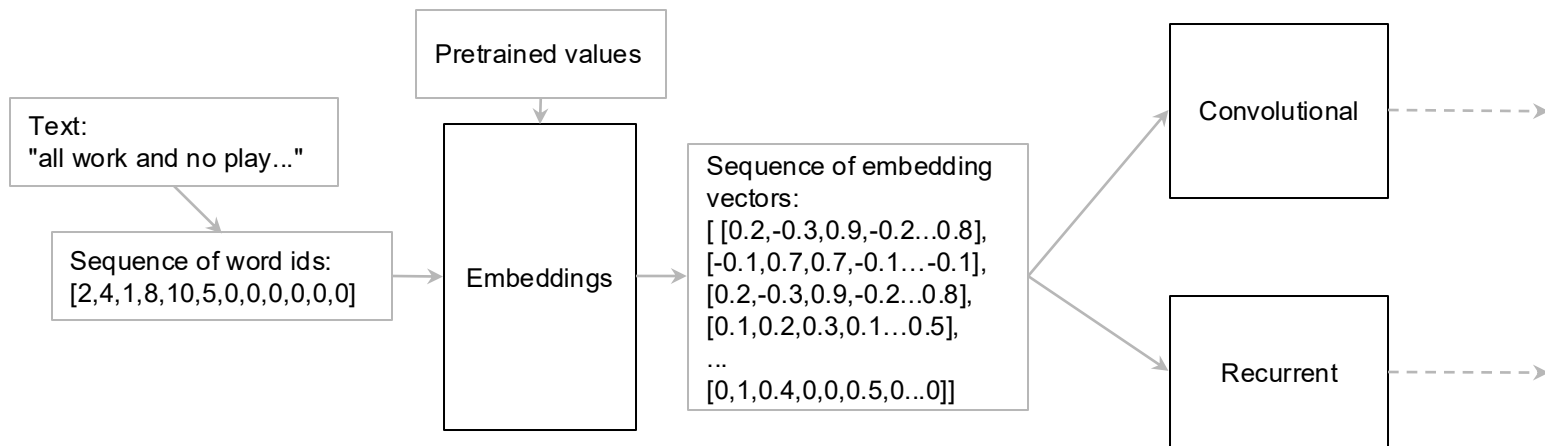
It consists of a matrix W of size $|F| \cdot n$, where n is the size of the embedding space.

It maps words to dense representations.

It can be initialized with random weights or *pretrained* embeddings.

During learning weights can be kept fixed (it makes sense only when using pretrained weights) or updated, to adapt embeddings to the task.

Embeddings in neural networks



Example:

https://github.com/fchollet/keras/blob/master/examples/imdb_cnn.py

Another example: <https://machinelearningmastery.com/use-word-embedding-layers-deep-learning-keras/>

Embeddings in NN: OOV words and padding

- LMs that use a vocabulary do not model **out-of-vocabulary words**.

Add a special *unknown* word (and embedding) for such words, to be learned during training.

- NNs usually process examples in *batches*, i.e., set of k examples.

Input sentences in a batch are required to be of the same length, so as to be processed in parallel.

For this reason a special **padding word** (and embedding) is added before/after (be consistent!) words of shorter sentence to match the length of the longest one.

The latest generation of language models

Recently, many competing deep networks have show outstanding abilities in language modeling.

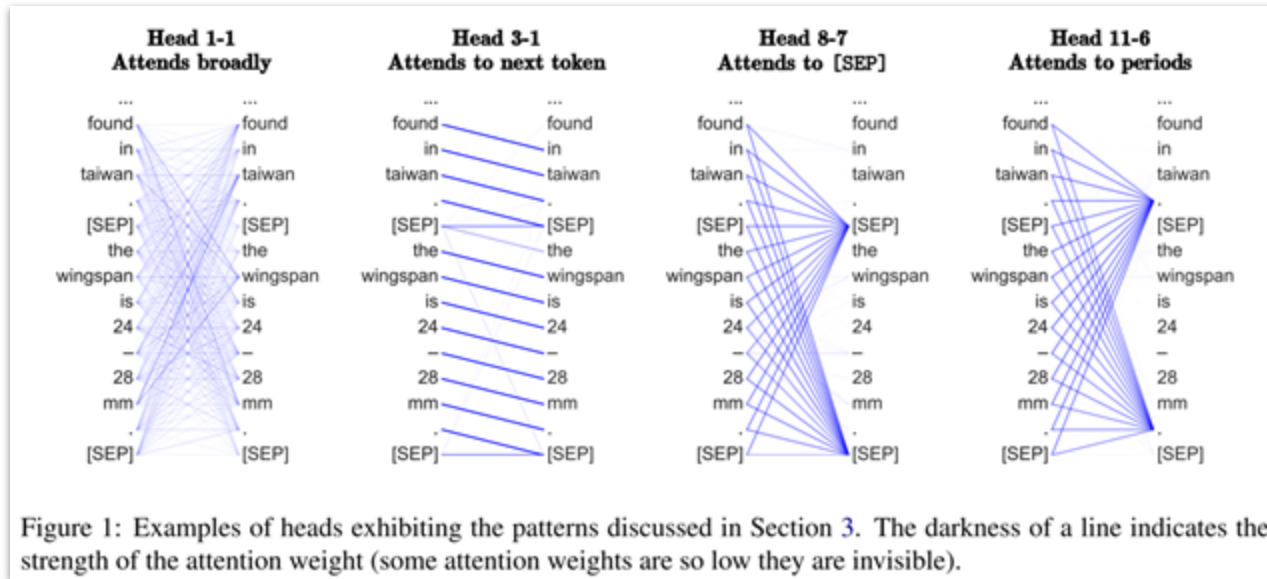
They exploit deep NN architectures such as bi-directional character level LSTM ([ELMo](#)) and **multi-headed self-attention** ([Transformer](#), [BERT](#) and [many others](#)).

They capture a lot of information about language, from huge collections of documents, into models that may have up to [billions of parameters](#).



Self-attention

Self-attention is a NN technique that learns the contribution of the contextual elements of a sequence to the definition of the semantic of a word.

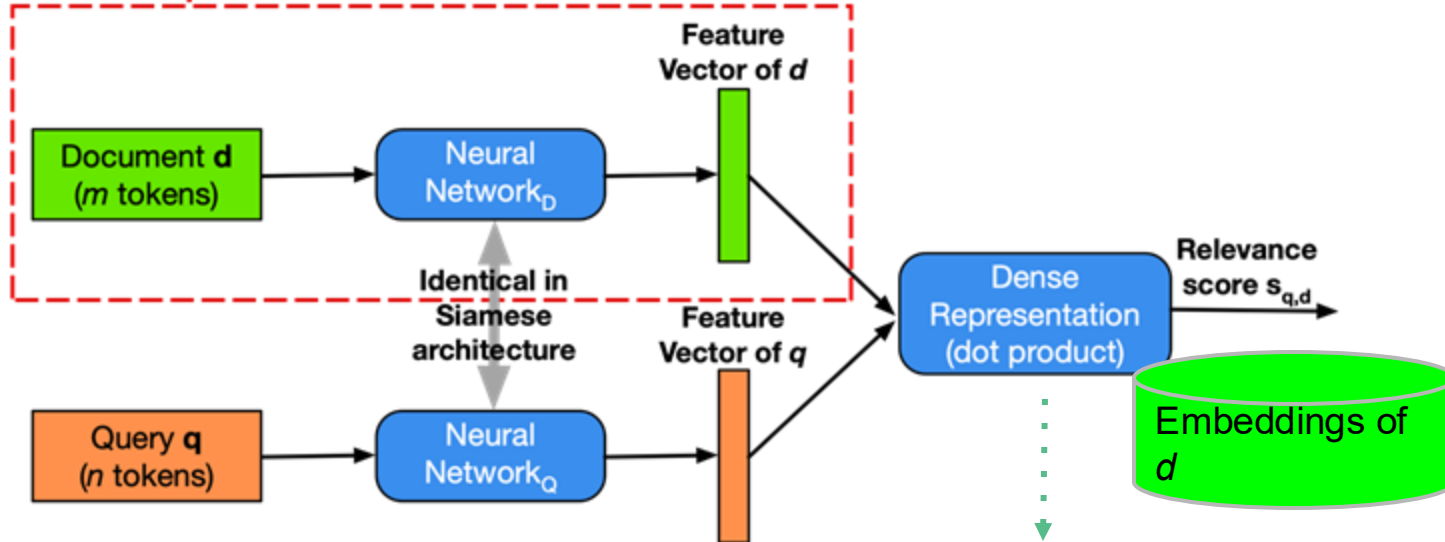


This process is similar to what [CNN filters learn from images](#).



Dense Representations and Retrieval

Offline Computation

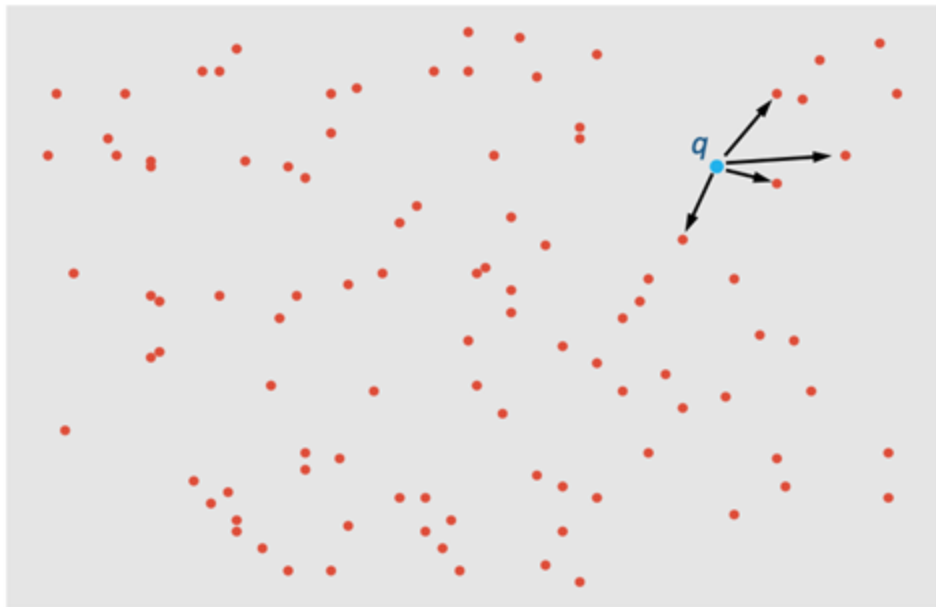


k Nearest Neighbour (kNN) Retrieval

- Closest k points in the N -dimensional space
- Exact vs. Approximate Nearest Neighbour
- Several techniques targeting efficiency

k Nearest Neighbour

- Retrieval as finding the most similar docs with the query
 - dot product
 - cosine similarity
- $O(n \log k)$
 - All d 's against q
 - Heap data structure that keeps k elements with higher similarity
- Slow/Exact vs Fast/Approximate
- Several techniques for ANN
 - Trees/Graphs/LSH/Quantization



Summary

Language models are a powerful tool to collect unsupervised information from a language or domain and to model it in machine-readable latent representations.

Such representations allow us to:

- explore a language, a domain, and discover its relevant entities and their syntactic and semantic relations.
- infuse general knowledge about a language or a domain into a supervised learning task, enabling the learned model to express a better generalization ability.